Messaging with SQS and SNS

Explore how to implement asynchronous messaging using Amazon Simple Queue Service and Amazon Simple Notification Service. Understand different queue and topic types, message delivery patterns, failure handling, integration with AWS Lambda, and best practices for building scalable, fault-tolerant serverless applications on AWS.

In this lesson, we'll explore Amazon Simple Queue Service (Amazon SQS) and Amazon Simple Notification Service (Amazon SNS) as tools for implementing scalable and decoupled application messaging. We examine their core features, messaging patterns, and how we can integrate these services into serverless architectures.

Introduction to messaging in AWS

Modern application architectures use asynchronous messaging to improve scalability, fault isolation, and decoupling. In AWS, Amazon SQS and Amazon SNS provide essential messaging functionality.

Amazon SQS is a pull-based queueing service that stores messages until consumers/receivers are ready to process them. It helps decouple microservices, batch processing systems, and other distributed components.

Amazon SNS is a push-based publish/subscribe (pub/sub) service that delivers messages to multiple subscribers. It supports multiple protocols such as HTTPS, Lambda, email, SMS, and Amazon SQS.

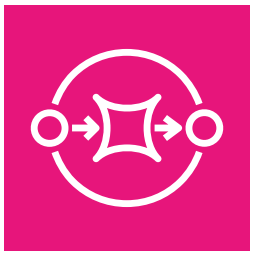
Ask

Asynchronous messaging via Amazon SQS and Amazon SNS contrasts with synchronous invocation patterns, such as using Amazon API Gateway to trigger

AWS Lambda. These patterns are used when durability, buffering, and loose coupling are needed. We can choose synchronous patterns when we need a response before continuing.

Amazon SQS: Message queues

Amazon Simple Queue Service (SQS) is a message queuing service that can help us decouple our application components. It uses queues to store the messages sent by producers and sends them to consumers using a polling model. Consumers check the queues for messages sent by the producer whenever they are ready to process them.



Amazon SQS uses redundant infrastructure and stores the messages it receives in multiple availability zones, making SQS queues highly available and durable. Unprocessed messages can be retained for a maximum of 14 days, after which all copies of the message are deleted from the SQS servers.

Ask

Redundant infrastructure in Amazon SQS

In SQS, it is the consumer's responsibility to delete the messages available in an SQS queue after processing them. Once a message “M1” is received by a consumer, SQS starts a visibility timeout period during which this message is not sent to any other consumer to avoid message reprocessing.

Types of SQS queues

In many scenarios, different applications have varying messaging needs; some prioritize strict ordering, while others focus on high throughput and at-least-once delivery. To address these needs, Amazon SQS offers multiple queue types tailored for different use cases. It supports two queue types:

1. Standard queues

In standard queues, messages are delivered at least once to each consumer. In this type of queue, the messages are stored in multiple availability zones, so there is a chance that multiple copies of one message are delivered. In some rare cases, we may delete a message whose copy is stored on an unavailable server. The message will become visible for processing once again when the server is available. Due to this, the order of delivery of the messages is not guaranteed in standard queues, and it is recommended to use them in applications where processing one message multiple times doesn’t affect it, such as sending email notifications or updating analytics counters.

Here's how we can create a standard queue using AWS CLI and AWS SDK:

AW...

```
aws sqs create-queue \
  --queue-name MyStandardQueue \
  --attributes VisibilityTimeout=30,MessageRetentionPeriod=1209600
```

Ask

Create a standard queue using AWS CLI

The `VisibilityTimeout=30` argument sets how long a message remains invisible after being received and the `MessageRetentionPeriod=1209600` argument retains unprocessed messages for 14 days.

Once we've created a standard queue, producers can begin sending messages to it. Standard queues offer at-least-once delivery and best-effort ordering. The following example shows how to send a message to a standard queue using AWS CLI and Boto3.

AW...

```
aws sqs send-message \
  --queue-url https://sqs.<Reegion>.amazonaws.com/<Account-ID>/MyStan
  --message-body "Hello from standard queue"
```

Ask

Send a message to a standard queue using AWS CLI

2. FIFO queues

In FIFO queues, messages are delivered to consumers exactly once, and their order of delivery is maintained. Due to this functionality, they are suitable for applications where the order of messages is critical, for example, e-commerce applications where the order of the messages being sent and delivered must be maintained.

While creating a FIFO queue, we must add the suffix `.fifo` in the queue's name. Here's how we can create one using AWS CLI and Python SDK:

AW...

```
aws sqs create-queue \
  --queue-name MyFifoQueue.fifo \
  --attributes FifoQueue=true
```

Create a FIFO queue using AWS CLI

The `FifoQueue=true` arguments enables FIFO behavior.

Once we've created a FIFO queue, we are ready to send messages to our queue. The code below shows how we can send messages to a FIFO queue using both the AWS CLI and the AWS Python SDK (Boto3), including the necessary parameters for group-based ordering and deduplication.

Ask

AW...

```
aws sqs send-message \
  --queue-url https://sqs.<Region>.amazonaws.com/<Account-ID>/MyFifoQu
  --message-body "This is a message for a FIFO queue" \
  --message-group-id "group-1" \
  --message-deduplication-id "unique-id-12345"
```

Send a message to a FIFO queue using AWS CLI

When sending a message to a FIFO queue, we must specify both a `MessageGroupId` and a `MessageDeduplicationId` (unless content-based deduplication is enabled). These arguments help us to set up the messages order and avoid duplication. Let's discuss them in detail:

- **Message deduplication ID:** This attribute helps to prevent message duplication in the queue and is provided by the sender. If a queue receives more than one message with the same message duplication ID within five minutes, it retains only the first one of these messages and discards all other.

> **Note:** There is another argument, `ContentBasedDeduplication`, which we can set to `true` to tell SQS to automatically generate the deduplication ID using a hash of the message body. This is useful when senders don't provide a `MessageDeduplicationId` themselves.

Ask

- **Message group ID:** This attribute is used to order the messages received by the queue. Messages with the same group ID are stored in the order they are

received and are then processed in order. While delivering messages, FIFO queues try to send as many messages as it can that belong to the same group, however, messages with different group IDs can be sent out of order.

Message grouping in FIFO queues

In the diagram above, there are two different message groups, M and X, sent by Producer 1 and 2, respectively. The message X1 is sent before the message M2; however, since the message group M was created before group X, SQS will send message M2 before it sends X1 to its consumers.

Choosing the right type of queue

To clarify the trade-offs and selection criteria between the two SQS queue types, the table below highlights the key differences between Standard and FIFO queues.

Feature	Standard Queue	FIFO Queue
Delivery Guarantee	At-least-once delivery	Exactly-once processing
Message Ordering	Best-effort ordering	Strict ordering within a message group
Throughput	Nearly unlimited	300 TPS (no batching), 3,000 TPS (with batching)
Deduplication	Not required	Required, via `MessageDeduplicati` content-based

Ask

Use Case Suitability	High-throughput, duplicates acceptable	Order-sensitive, duplicates must be avoided

These distinctions across delivery guarantees, ordering behavior, throughput capacity, deduplication requirements, and use case suitability help us choose the right queue type based on our application's needs.

SQS message failure handling

When a consumer (such as an EC2 worker or Lambda function) receives a message from SQS, it must explicitly delete the message to confirm successful processing. If it fails to do so, the message becomes visible again after the visibility timeout expires.

Each time a message is received (whether successfully processed or not), the `ReceiveCount` increases. If the number of receives exceeds the configured `maxReceiveCount`, SQS automatically routes the message to a Dead Letter Queue (DLQ). A dead-letter queue is a queue where we can send all unconsumed messages. Through this, we can isolate these messages and try to determine why their processing failed.

To enable DLQ behavior, we first create a dead-letter queue with a redrive policy and then we associate this dead letter queue with a source queue. A redrive policy specifies how the DLQ interacts with the source queue. Here's a sample redrive policy:

index.yaml YAML

```
{
  "deadLetterTargetArn": "arn:aws:sqs:<Region>:<Account-ID>:my-dlq",
  "maxReceiveCount": "5"
}
```

Ask

Sample redrive policy

In this example, any message that is received more than 5 times without being deleted will be moved to the queue identified by `deadLetterTargetArn`.

Amazon SNS: Message broadcasting

Amazon Simple Notification Service (SNS) is a message delivery service provided by Amazon that delivers messages from a publisher to a subscriber. It uses the publish-subscribe model to deliver these messages. In this model, a publisher sends a message to a topic, which is then delivered to all of its subscriber endpoints. Since the publishers and subscribers don't need to know about each other, Amazon SNS can be used to decouple the components of a distributed system and deliver messages between them.

Ask

Publish/subscribe model

In Amazon SNS, a topic is a communication channel that receives messages from a publisher and delivers them to all available subscriber endpoints.

Types of SNS topics

Like SQS queues, Amazon SNS offers different topic types to support varied messaging needs and delivery guarantees. SNS topics can be of two types:

1. Standard topic

In standard topics, messages are sent to the subscribers at least once, and there is no mechanism to preserve the order of the messages received from the topic's producers. Other than this, in standard topics, there is a chance that message duplicates exist, as a producer might send the same message multiple times to the topic.

We can create a standard SNS topic using the following CLI command:

Ask

AW...

```
aws sns create-topic \
```

```
  --name MyStandardTopic
```

Create a standard topic using AWS CLI

Once a standard topic is created, applications can begin publishing messages to it. SNS then pushes these messages to all subscribed endpoints, which may include Lambda functions, SQS queues, email addresses, or HTTP/HTTPS webhooks. Standard topics support high throughput, best-effort ordering, and at-least-once delivery, making them ideal for broadcasting non-critical messages to multiple consumers. Here's how we can publish a message to an SQS topic:

AW...

```
aws sns publish \
  --topic-arn arn:aws:sns:<Region>:<Account-ID>:MyStandardTopic \
  --message "Hello from SNS"
```

Ask

Send a message to a standard topic using AWS CLI

Standard topics are used in applications where message ordering is not critical and requires a high throughput.

2. FIFO topic

We can create FIFO (first in, first out) topics in Amazon SNS. In these topics, messages are sent to subscribers exactly once, and messages are delivered to the subscriber endpoints in the order they were received. These topics use a deduplication ID. In case multiple messages are received with the same deduplication ID, within five minutes, only the first message is forwarded to the topic's subscribers.

> **Note:** FIFO topics are not supported for all protocols (like email or SMS), they are compatible with AWS Lambda, HTTP/HTTPS endpoints, and SQS FIFO queues.

To create a FIFO SNS topic, we can use the following CLI and SDK commands:

AW...

```
aws sns create-topic \
  --name MyFifoTopic.fifo \
  --attributes FifoTopic=true,ContentBasedDeduplication=true
```

Ask

Create a FIFO topic using AWS CLI

> **Note:** We need to provide the suffix `.fifo` in our topics' name in case we want to create a FIFO topic.

The `FifoQueue=true` arguments enables FIFO behavior and the `ContentBasedDeduplication=true` argument automatically generates deduplication IDs from the message body, avoiding duplicates.

Publishing to a FIFO topic is slightly different. These topics enforce exactly-once message delivery and strict ordering, but only to FIFO SQS queues (other endpoint types are not supported). Just like for FIFO queues, when publishing a message, we must include a `MessageGroupId` and a `MessageDeduplicationId` (unless content-based deduplication is enabled on the topic). These parameters ensure the integrity of order and prevent duplication. Here's how we can publish a message to a FIFO topic:

AW...

```
aws sns publish \
  --topic-arn arn:aws:sns:<Region>:<Account-ID>:MyFifoTopic.fifo \
  --message "Order received: 12345" \
  --message-group-id "order-processing-group" \
  --message-deduplication-id "order12345"
```

Ask

Send a message to a FIFO topic using AWS CLI

These topics are used in applications where the order of messages being sent and received is critical, and low throughput can be tolerated.

Choosing the right type of topic

Choosing the right type of topic based on our needs depends on the required message delivery semantics and ordering guarantees. The table below summarizes the key differences between Standard and FIFO topics in Amazon SNS:

Feature	Standard Topic	FIFO Topic
Message Ordering	Best-effort (no guaranteed order)	Strict ordering based on message group ID
Delivery Attempts	At-least-once	Exactly-once
Duplication Handling	Possible duplicates	Automatic deduplication using MessageDeduplicationId or content-based deduplication
Throughput	Nearly unlimited	Up to 300 TPS (or 3,000 with batching)
Supported Protocols	All (Lambda, SQS, HTTP/S, email, etc.)	SQS FIFO queues and Lambda (with restrictions)
Use Case Suitability	High-throughput, loosely ordered fan-out	Financial transactions, logs, or any strict-order workflows

These differences help us decide when to favor consistency and ordering (FIFO) versus scalability and flexibility (Standard).

SNS failure handling

Amazon SNS uses push-based delivery, which introduces several protocol-specific failure modes. For instance, HTTP/HTTPS endpoints may return 4 5xx errors due to issues like authentication failure or server overload. Lambda function subscribers might time out, get throttled, or encounter runtime

Ask

exceptions during execution. In the case of email or SMS delivery, messages can be throttled or rejected due to provider-specific constraints or invalid recipient addresses.

SNS retries failed deliveries using exponential backoff. This retry logic varies across protocols:

- **HTTP/HTTPS endpoints:** SNS retries delivery for a few hours (usually up to 23 hours), adjusting intervals between attempts.
- **Lambda functions:** The retries occur for up to 6 hours, regardless of the error type. After that, the message is discarded unless we've configured a DLQ on the Lambda function.

> **Note:** SNS does not support attaching DLQs directly to topics. However, message retention for failed deliveries can still be achieved using the downstream services.

- **Email/SMS:** Retry logic is not always consistent. For email, Simple Email Service (SES) handles retries independently, and for SMS, retries depend on carrier feedback, which SNS doesn't expose in detail.

Optimizing delivery with filters

SNS subscription filter policies allow selective message delivery based on message attributes. It support conditions such as `StringEquals`, `StringLike`, `Numeric`, and `AnythingBut`. These enable precise targeting of messages to specific subscribers. For example, a Lambda subscriber can be configured to only receive messages where the eventType attribute equals `"OrderCreated"`.

To implement a filter policy on an SNS subscription using the AWS CLI, we use the following command:

Ask

main.sh Shell

```
aws sns set-subscription-attributes \
  --subscription-arn arn:aws:sns:<Region>:<Account-ID>:OrderTopic:<To
  --attribute-name FilterPolicy \
  --attribute-value '{"eventType": ["OrderCreated"]}'
```

SNS subscription filter policy

In this example, only messages published to the topic with the message attribute `eventType` equal to `OrderCreated` will be delivered to the specified subscription.

By filtering out irrelevant messages, these policies help reduce downstream invocations, lower costs, and improve overall system efficiency. This configuration ensures that the subscriber only receives relevant messages, making the system more efficient and reliable.

Integration patterns

Amazon SNS and Amazon SQS work well together to implement scalable, fault-tolerant messaging workflows.

> **Note:** While Amazon SQS and Amazon SNS support asynchronous messaging, Amazon Kinesis and Amazon EventBridge are preferable when we need near real-time streaming, event replay, or complex event routing

Ask

Let's discuss a couple of integration patterns of these services:

Fanout pattern

A single message sent to an SNS topic can be delivered to multiple SQS queues. Each queue can then be processed independently by different consumers or Lambda functions.

AWS Lambda integration

AWS Lambda integrates with both Amazon SNS and Amazon SQS, though the mechanisms differ. With SNS, the integration is push-based. When a message is published to a topic, SNS directly invokes the Lambda function. This is configured by subscribing the function to the topic. We can do that using the CLI and SDK commands given below:

AW...

```
aws sns subscribe \
  --topic-arn arn:aws:sns:<Region>:<Account-ID>:OrderTopic \
  --protocol lambda \
  --notification-endpoint arn:aws:lambda:<Region>:<Account-ID>:functi
```

SNS integration with Lambda using AWS CLI

SQS integration, by contrast, uses a poll-based model. With SQS integration, Lambda doesn't passively wait for messages. Instead, it actively polls the queue at regular intervals using an event source mapping, and invokes the function whenever messages are found. We can create and configure these event source mappings using the following CLI and SDK commands:

Ask

AW...

```
aws lambda create-event-source-mapping \
  --function-name myLambdaFunction \
  --event-source-arn arn:aws:sqs:<Region>:<Account-ID>:OrderQueue \
  --batch-size 10 \
  --maximum-batching-window-in-seconds 5 \
  --maximum-retry-attempts 3
```

SQS integration with Lambda using AWS CLI

There are three important arguments in the commands above which provide fine-grained control over throughput, latency, and error recovery:

- `--batch-size` controls how many messages are sent to the function at once, up to a maximum of 10 for standard queues and 10 for FIFO queues.
- `--maximum-batching-window-in-seconds` allows Lambda to wait up to the specified time to gather a full batch.
- `--maximum-retry-attempts` sets how many times Lambda retries a failed batch before sending it to a dead-letter queue, if configured.

While SNS enables instant fan-out to multiple subscribers, SQS with Lambda offers more controlled, resilient processing making them ideal for high-volume or delayed workloads.

Required IAM permissions for message flo

Ask

To ensure proper message flow between services, specific IAM permissions are required. These permissions must be granted to the publishing and consuming

services or their associated IAM roles:

SNS to SQS

When SNS delivers messages to an SQS queue, the SNS service principal (`sns.amazonaws.com`) must have permission to call `sqs:SendMessage` on the target queue. This is enforced through an SQS access policy, not an IAM role. Here's a sample access policy we can use:

index.yaml YAML

```
{
  "Effect": "Allow",
  "Principal": {
    "Service": "sns.amazonaws.com"
  },
  "Action": "sqs:SendMessage",
  "Resource": "arn:aws:sqs:<Region>:<Account-ID>:<queue-name>",
  "Condition": {
    "ArnEquals": {
      "aws:SourceArn": "arn:aws:sns:<Region>:<Account-ID>:<topic-name>
    }
  }
}
```

Sample IAM policy

> **Note:** Without this, SNS will silently fail to deliver messages to SQS, even if we have subscriptions set up correctly.

Lambda polling SQS

When a Lambda function polls messages from SQS, the Lambda executio must have these permissions:

Ask

- `sqs:ReceiveMessage`—to poll messages

- `sqs:DeleteMessage`—to remove messages after processing
- `sqs:GetQueueAttributes`—to read attributes like visibility timeout (optional but recommended)

Given below is a sample IAM policy that we can attach with the Lambda execution role:

index.yaml YAML

```yaml
{
  "Effect": "Allow",
  "Action": [
    "sqs:ReceiveMessage",
    "sqs:DeleteMessage",
    "sqs:GetQueueAttributes"
  ],
  "Resource": "arn:aws:sqs:<Region>:<Account-ID>:<queue-name>"
}
```

Sample IAM policy

These permissions are required for successful event source mapping when Lambda is configured to trigger from SQS.

Reliability and monitoring

To enhance reliability and observability, we can do the following:

- Monitor Lambda throttling when consuming from SQS. High message volume or concurrent invocations may exceed the function's concurre limit. We can set reserved concurrency to cap execution or configure alarms to trigger scaling actions.

Ask

- Configure DLQs to isolate and examine failed messages.
- Apply exponential backoff with jitter to retry operations without overwhelming systems. AWS SDKs handle this automatically by default.
- Use Amazon CloudWatch to monitor queue depth, message age, and failed executions. Here are some key metrics for SQS and SNS:
    - `ApproximateAgeOfOldestMessage` (SQS)**:** This detects backlog issues, if this rises, messages are waiting too long.

Ask